# NetNoise: Un Beacon di Casualità Pubblica Decentralizzato Basato su Proof-of-Work Memory-Hard con Chain-Linking Crittografico

Giorgio Martucci
*Università degli Studi [Nome]*
giorgio.martucci@example.edu

Febbraio 2026

**Sommario**

Presentiamo NetNoise, un sistema decentralizzato per la generazione di casualità pubblica verificabile che combina entropia fisica, Proof-of-Work memory-hard e un meccanismo di chain-linking crittografico attraverso l'uso di Argon2id come funzione di consenso. L'innovazione principale consiste nell'utilizzo dell'hash del blocco precedente come parametro salt di Argon2, creando una dipendenza matematica esplicita tra blocchi consecutivi che garantisce immutabilità computazionale.

**Parole chiave:** Casualità Pubblica, Blockchain, Proof-of-Work, Funzioni Memory-Hard, Chain-Linking Crittografico, Argon2, HMAC-DRBG

## Indice

**1 Introduzione** **3**

**2 Protocollo di Consenso e Struttura della Blockchain** **3**
2.1 Modello di Sistema . . . 3
2.2 Struttura del Blocco . . . 3
2.3 Consenso tramite Proof-of-Work . . . 4
2.3.1 Difficoltà e Target . . . 4
2.3.2 Aggiustamento Dinamico della Difficoltà . . . 4
2.4 Regola di Fork Resolution . . . 4

**3 Meccanismo di Generazione della Casualità** **5**
3.1 Panoramica del Protocollo . . . 5
3.2 Fase 1: Raccolta Entropia Iniziale . . . 5
3.2.1 Sorgenti di Entropia del Sistema Operativo . . . 5
3.3 Fase 2: Mining Proof-of-Work con Chain-Linking Crittografico . . . 6

3.3.1 Algoritmo di Mining . . . . . 6
3.3.2 Parametri Argon2id . . . . . 6
3.4 Fase 3: Estrazione della Casualità Finale . . . . . 7
3.4.1 Combinazione delle Sorgenti . . . . . 7
3.4.2 Espansione HMAC-DRBG . . . . . 8
3.5 Riepilogo del Flusso Completo . . . . . 9

# 1 Introduzione

In questo lavoro presentiamo NetNoise, un nuovo protocollo per la generazione decentralizzata di casualità pubblica che elimina completamente la necessità di trusted parties. I contributi principali sono:

1. **Chain-Linking Crittografico via Salt:** Introduciamo un meccanismo in cui l'hash del blocco precedente viene utilizzato come parametro salt della funzione Proof-of-Work (Argon2id). Questo crea una dipendenza matematica esplicita tra blocchi consecutivi, garantendo immutabilità computazionale più forte rispetto al chain-linking tradizionale di Bitcoin.

2. **Architettura Multi-Livello per la Casualità:** Combiniamo entropia fisica dal sistema operativo, casualità derivata dal processo di mining (nonce vincente), e espansione crittografica tramite HMAC-DRBG, creando un generatore ibrido che mantiene sia imprevedibilità fisica che verificabilità matematica.

# 2 Protocollo di Consenso e Struttura della Blockchain

## 2.1 Modello di Sistema

Consideriamo una rete peer-to-peer distribuita composta da $n$ nodi (miner) connessi secondo un modello di rete parzialmente sincrono con delay massimo $\Delta$. Ogni nodo mantiene una copia locale della blockchain e partecipa al mining.

## 2.2 Struttura del Blocco

**Definizione 2.1** (Blocco NetNoise)**.** Un blocco $B_N$ nella blockchain NetNoise è una tupla:

$$B_N = (N, t_N, s_N, n_N, h_{N-1}, h_N, R_N, D_N)$$

dove:

- $N \in \mathbb{N}$ è l'indice del blocco (altezza)
- $t_N \in \mathbb{N}$ è il timestamp Unix
- $s_N \in \{0,1\}^{512}$ è il seed di entropia iniziale
- $n_N \in \mathbb{N}$ è il nonce vincente trovato dal mining
- $h_{N-1} \in \{0,1\}^{256}$ è l'hash del blocco precedente
- $h_N \in \{0,1\}^{256}$ è l'hash Argon2id di questo blocco
- $R_N \in \{0,1\}^{256}$ è la casualità finale del blocco
- $D_N \in \mathbb{N}$ è il parametro di difficoltà

## 2.3 Consenso tramite Proof-of-Work

Il consenso in NetNoise segue il paradigma Nakamoto: i nodi competono per trovare soluzioni a un puzzle computazionale e il blocco valido propagato più rapidamente viene accettato dalla rete.

### 2.3.1 Difficoltà e Target

Dato un parametro di difficoltà $D_N \in \mathbb{N}$, definiamo il target numerico come:

$$T_N = \frac{2^{256}}{D_N} \tag{1}$$

Un hash $h_N$ è considerato valido se:

$$\text{int}(h_N) < T_N \tag{2}$$

dove $\text{int}(\cdot)$ interpreta i 256 bit come un intero senza segno.

### 2.3.2 Aggiustamento Dinamico della Difficoltà

Per mantenere un tempo medio di blocco target $\tau_{\text{target}}$, la difficoltà viene ricalcolata ogni $E$ blocchi (epoch) secondo:

$$D_{N+E} = D_N \times \frac{\tau_{\text{actual}}}{\tau_{\text{target}}} \tag{3}$$

dove:

$$\tau_{\text{actual}} = \frac{1}{E} \sum_{i=N}^{N+E-1} (t_i - t_{i-1}) \tag{4}$$

è il tempo medio effettivo tra blocchi nell'ultimo epoch.

## 2.4 Regola di Fork Resolution

In caso di fork (blocchi concorrenti alla stessa altezza), i nodi adottano la catena con **peso cumulativo maggiore**:

**Definizione 2.2** (Peso della Catena)**.** Il peso di una catena $C = [B_0, B_1, \ldots, B_N]$ è definito come:

$$W(C) = \sum_{i=0}^{N} D_i$$

Questa regola (analoga al "longest chain" di Bitcoin ma pesata per difficoltà) garantisce che la catena con più lavoro computazionale cumulativo prevalga.

# 3 Meccanismo di Generazione della Casualità

Il cuore di NetNoise è il processo di generazione della casualità, che avviene in tre fasi distinte: raccolta entropia, mining con chain-linking, ed estrazione finale.

## 3.1 Panoramica del Protocollo

Il protocollo di generazione della casualità può essere formalizzato come la composizione di tre funzioni:

$$\mathcal{R} = \mathsf{Estrai} \circ \mathsf{PoW} \circ \mathsf{Entropia} \tag{5}$$

dove:

- $\mathsf{Entropia} : \emptyset \to \{0,1\}^{512}$ genera il seed iniziale
- $\mathsf{PoW} : \{0,1\}^{512} \times \{0,1\}^{256} \to \mathbb{N} \times \{0,1\}^{256}$ esegue il mining
- $\mathsf{Estrai} : \{0,1\}^* \to \{0,1\}^{256}$ produce la casualità finale

## 3.2 Fase 1: Raccolta Entropia Iniziale

Ogni miner, prima di iniziare il mining di un nuovo blocco, genera localmente un seed di entropia:

$$s_N \leftarrow \texttt{os.urandom}(64) \tag{6}$$

dove $s_N \in \{0,1\}^{512}$ è un vettore di 64 byte (512 bit).

### 3.2.1 Sorgenti di Entropia del Sistema Operativo

La primitiva `os.urandom` sui sistemi Unix-like legge dal dispositivo virtuale `/dev/urandom`, che accumula entropia da molteplici sorgenti hardware:

1. **Interrupt hardware:** Timing imprevedibili di eventi I/O (tastiera, mouse, disco, rete)
2. **Rumore termico:** Fluttuazioni nei sensori di temperatura dei componenti
3. **Jitter del timer:** Variazioni nei cicli di clock del processore
4. **Eventi di scheduling:** Decisioni dello scheduler del kernel
5. **Istruzioni CPU dedicate:** RDRAND/RDSEED (se disponibili)

Queste sorgenti sono intrinsecamente non deterministiche e resistenti alla manipolazione remota, fornendo una solida base di entropia fisica per il sistema.

### 3.3 Fase 2: Mining Proof-of-Work con Chain-Linking Crittografico

Il mining in NetNoise non è solo un meccanismo di consenso, ma contribuisce direttamente alla casualità finale attraverso due elementi: il nonce vincente e l'innovativo schema di chain-linking.

#### 3.3.1 Algoritmo di Mining

Dato il seed $s_N$ e l'hash del blocco precedente $h_{N-1}$, il processo di mining cerca un nonce $n$ che soddisfi il criterio di difficoltà.

**Algorithm 1** Mining con Chain-Linking Crittografico

**Require:** $s_N$ (seed), $h_{N-1}$ (hash precedente), $D_N$ (difficoltà)
**Ensure:** $(n_N, h_N)$ (nonce vincente, hash blocco)

```
// CHAIN-LINKING: Salt derivato dal blocco precedente
salt_N ← h_{N-1}[0 : 16]                          ▷ Primi 16 byte di h_{N-1}
T_N ← 2^256 / D_N                                 ▷ Calcola target
n ← 0                                             ▷ Inizializza nonce
while TRUE do
    input ← s_N ‖ n.to_bytes(8)                   ▷ Concatena seed e nonce
    // Hash con Argon2id usando salt dal blocco precedente
    h ← Argon2id(input, salt_N, t = 3, m = 65536, p = 1)
    h_int ← bytes_to_int(h)                       ▷ Interpreta come intero
    if h_int < T_N then                           ▷ Verifica se soddisfa difficoltà
        return (n, h)                             ▷ Soluzione trovata!
    end if
    n ← n + 1                                     ▷ Incrementa nonce e riprova
end while
```

**L'elemento distintivo di NetNoise è l'uso dell'hash del blocco precedente come parametro salt della funzione Argon2id.** Questo crea una dipendenza matematica esplicita e indistruttibile tra blocchi consecutivi.

Formalmente, per il blocco $N$:

$$\text{salt}_N = h_{N-1}[0:16] \tag{7}$$

dove $h_{N-1}[0:16]$ denota i primi 16 byte dell'hash del blocco precedente.

Di conseguenza, l'hash del blocco $N$ è calcolato come:

$$h_N = \textsc{Argon2id}(s_N \| n_N, \text{salt} = h_{N-1}[0:16], t = 3, m = 65536, p = 1) \tag{8}$$

Questa costruzione ha implicazioni profonde sulla sicurezza, come dimostreremo nella Sezione **??**.

#### 3.3.2 Parametri Argon2id

Utilizziamo Argon2id (variante ibrida) con i parametri riportati nella Tabella 1.

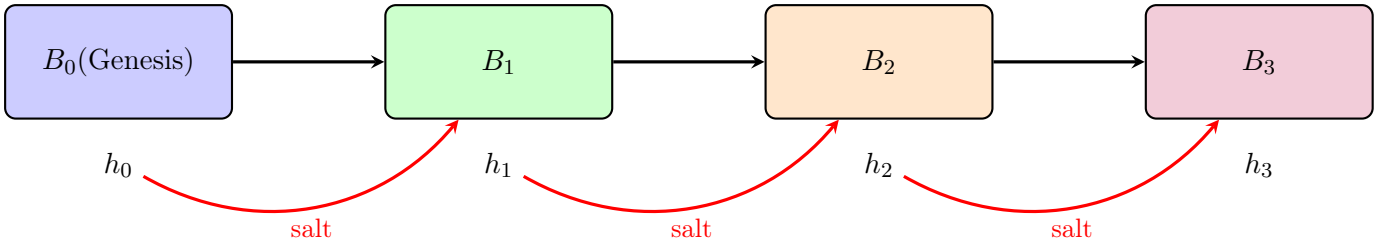


Figura 1: Chain-linking crittografico in NetNoise. L'hash di ogni blocco viene utilizzato come parametro salt per il calcolo Argon2id del blocco successivo (frecce rosse), creando una dipendenza matematica esplicita.

Tabella 1: Parametri Argon2id per il Proof-of-Work di NetNoise

| **Parametro** | **Valore** | **Motivazione** |
|---|---|---|
| `time_cost` ($t$) | 3 | Bilanciamento tra velocità e sicurezza; 3 iterazioni sufficienti per memory-hardness |
| `memory_cost` ($m$) | 65536 KiB (64 MB) | Garantisce resistenza ASIC richiedendo banda di memoria significativa |
| `parallelism` ($p$) | 1 | Mining sequenziale per evitare vantaggi da parallelizzazione massiva |
| `hash_len` | 32 byte | Output 256-bit compatibile con sicurezza crittografica standard |
| `type` | Argon2id | Variante ibrida resistente sia a timing attacks che a trade-off attacks |

Con questi parametri, su CPU consumer moderna (Intel Core i7-10700K @ 3.8 GHz):

- **Throughput:** ∼50–100 hash/secondo
- **Tempo atteso per blocco:** ∼10–20 secondi (con difficoltà adeguata)
- **Memoria richiesta:** 64 MB RAM per hash

Questo rende economicamente non conveniente lo sviluppo di ASIC specializzati, preservando la decentralizzazione del mining.

## 3.4 Fase 3: Estrazione della Casualità Finale

Una volta completato il mining e determinati il nonce vincente $n_N$ e l'hash $h_N$, procediamo all'estrazione della casualità finale.

### 3.4.1 Combinazione delle Sorgenti

Combiniamo tutte le sorgenti di entropia disponibili:

$$\text{entropy}_N = s_N \| n_N \| h_N \tag{9}$$

dove:

- $s_N \in \{0,1\}^{512}$ è l'entropia iniziale dal sistema operativo
- $n_N \in \mathbb{N}$ è il nonce vincente (convertito in 8 byte)
- $h_N \in \{0,1\}^{256}$ è l'hash Argon2id del blocco

La lunghezza totale è:

$$|\text{entropy}_N| = 512 + 64 + 256 = 832 \text{ bit} \tag{10}$$

### 3.4.2 Espansione HMAC-DRBG

Per ottenere la casualità finale, utilizziamo HMAC-DRBG (Deterministic Random Bit Generator basato su HMAC) come specificato in NIST SP 800-90A [**?**].

**Definizione 3.1** (Casualità del Blocco)**.** La casualità del blocco $B_N$ è definita come:

$$R_N = \text{HMAC-DRBG.}\mathsf{Generate}(\text{entropy}_N, 32) \tag{11}$$

dove HMAC-DRBG.$\mathsf{Generate}$ è la funzione di generazione HMAC-DRBG che produce 32 byte di output.

**Funzionamento di HMAC-DRBG.** HMAC-DRBG mantiene uno stato interno $(K, V)$ dove $K, V \in \{0,1\}^{256}$.

La funzione di update è:

**Algorithm 2** HMAC-DRBG Update

```
procedure UPDATE(K, V, provided_data)
    K ← HMAC_K(V ‖ 0x00 ‖ provided_data)
    V ← HMAC_K(V)
    if provided_data ≠ ϵ then
        K ← HMAC_K(V ‖ 0x01 ‖ provided_data)
        V ← HMAC_K(V)
    end if
    return (K, V)
end procedure
```

La generazione procede iterativamente:

**Algorithm 3** HMAC-DRBG Generate

```
procedure GENERATE(requested_bytes)
    output ← ϵ
    while |output| < requested_bytes do
        V ← HMAC_K(V)
        output ← output‖V
    end while
    (K, V) ← UPDATE(K, V, ϵ)                    ▷ Forward secrecy
    return output[0 : requested_bytes]
end procedure
```

**Forward Secrecy.** L'aggiornamento finale dello stato dopo ogni generazione garantisce che, anche se lo stato $(K, V)$ venisse compromesso in un momento futuro, gli output precedenti rimangono computazionalmente indistinguibili dal random.

### 3.5 Riepilogo del Flusso Completo

La Figura 2 riassume il processo end-to-end.

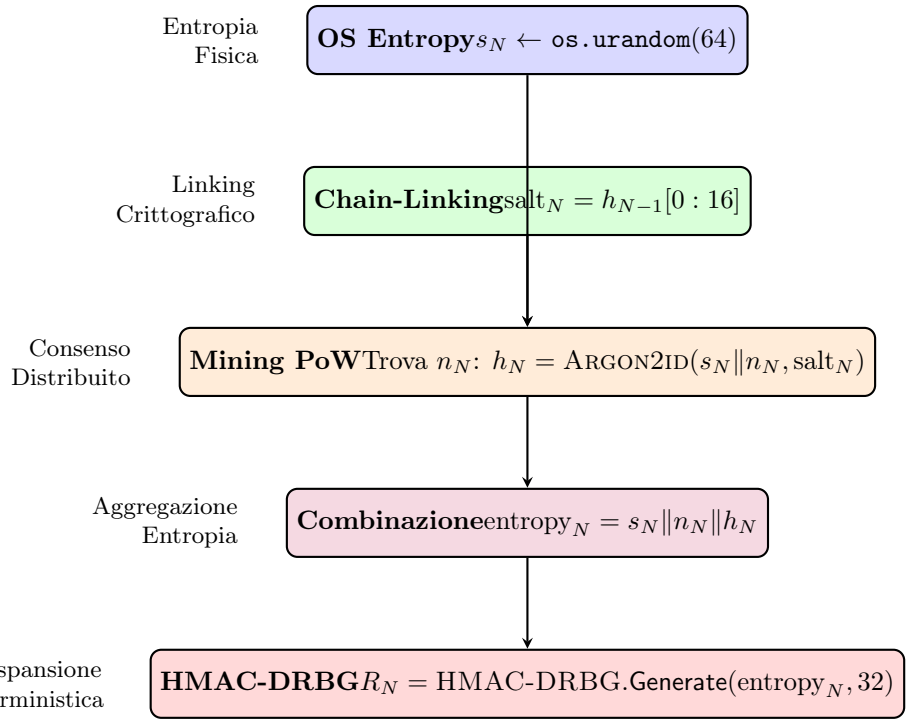


Figura 2: Flusso completo della generazione di casualità in NETNOISE. Ogni fase contribuisce con proprietà di sicurezza specifiche al sistema complessivo.